

Data Module for BG-Forecasting

This module provides comprehensive data loading, preprocessing, and PyTorch integration for blood glucose forecasting research. The implementation is **based on and adapted from the GluPred project**, with significant enhancements for research workflows and modern deep learning practices.

Overview

The data module follows a clean separation of concerns:

- **Loaders:** Extract raw data from XML files without modifications
- **Preprocessors:** Handle all data transformations, feature engineering, and quality control
- **PyTorch Datasets:** Convert preprocessed data into training-ready tensors

Module Structure

```
benchmark/data/  
├── __init__.py          # Module initialization  
├── loaders.py           # Raw data loading from XML files  
├── preprocessors.py     # Data preprocessing and feature engineering  
├── torch_dataset.py     # PyTorch dataset classes for training  
└── README.md           # This documentation
```

Data Flow Pipeline

```
Raw OhioT1DM XML Files → Loader → Preprocessor → PyTorch Dataset → Model  
Training
```

1. **Loading:** XML files are parsed and combined into raw DataFrames
2. **Preprocessing:** Missing values, outliers, and features are handled
3. **Dataset Creation:** Time series sequences are extracted for training
4. **Model Training:** PyTorch DataLoaders feed data to neural networks

Supported Datasets

OhioT1DM Dataset

The primary supported dataset is the OhioT1DM (Ohio Type 1 Diabetes Mellitus) dataset from the Ohio University Diabetes Research Center.

Versions Supported:

- **2018:** 6 patients, training/test splits

- **2020:** 6 patients, training/test splits

Data Types:

- Continuous Glucose Monitoring (CGM)
 - Insulin delivery (basal and bolus)
 - Meal information (carbohydrates, meal type)
 - Physiological data (heart rate, galvanic skin response, skin temperature)
 - Lifestyle data (sleep, work, exercise)
-

Core Components

1. Data Loaders (`loaders.py`)

`OhioT1DMDataloader`

Purpose: Parse OhioT1DM XML files and return raw, unmodified DataFrames.

Key Features:

- Handles both 2018 and 2020 dataset versions
- Parses XML files for CGM, insulin, meals, and physiological data
- Combines multiple data sources with proper time alignment
- Returns raw data without any preprocessing

Usage:

```
from benchmark.data.loaders import OhioT1DMDataloader

loader = OhioT1DMDataloader(
    data_dir="/path/to/data",
    sampling_rate=5, # 5-minute intervals
    version="2020"
)

# Load single patient
patient_df = loader.load_patient_data(540, 'train')

# Load multiple patients
train_data = load_ohiot1dm_data(
    data_dir="/path/to/data",
    patient_ids=[540, 544, 552],
    mode='train',
    version='2020'
)
```

Columns Returned:

- **glucose**: Blood glucose levels (mg/dL)
- **basal**: Basal insulin rate (units/hour)
- **bolus**: Bolus insulin (units)
- **bolus_dur**: Bolus duration (minutes)
- **carbs**: Carbohydrate intake (grams)
- **meal_type**: Type of meal
- **hr**: Heart rate (bpm)
- **gsr**: Galvanic skin response (μ S)
- **st**: Skin temperature ($^{\circ}$ C)

2. Data Preprocessors (**preprocessors.py**)

OhioBGDataPreprocessor

Purpose: Transform raw data into ML-ready format with comprehensive preprocessing.

Key Features:

- **Missing Data Handling:** Fill missing values, handle -1 markers
- **Temporal Event Processing:** Apply insulin duration, meal effects
- **Feature Engineering:** Time-based features, glucose derivatives, cumulative features
- **Outlier Detection:** Statistical outlier identification and flagging
- **Normalization:** Multiple scaling options (standard, minmax, robust)
- **Sequence Creation:** Extract input-output pairs for forecasting

Processing Pipeline:

1. **Basic Preprocessing:** Handle missing values, temporal events
2. **Feature Engineering:** Create time-based and glucose-derived features
3. **Outlier Detection:** Flag anomalous values
4. **Normalization:** Scale features for ML models
5. **Sequence Creation:** Extract training sequences

Usage:

```
from benchmark.data.preprocessors import OhioBGDataPreprocessor

preprocessor = OhioBGDataPreprocessor(
    target_column='glucose',
    sampling_rate=5
)

# Full preprocessing pipeline
processed_df = preprocessor.preprocess_patient_data(
    raw_df,
    apply_basic_preprocessing=True,
    include_feature_engineering=True,
    normalize=True,
    handle_missing='interpolate'
```

```
)

# Create sequences for training
X, y = preprocessor.create_sequences(
    processed_df,
    sequence_length=12,      # 1 hour history
    prediction_horizon=6,    # 30 minutes ahead
    step_size=1
)
```

Feature Engineering:

- **Temporal Features:** Hour, day of week, weekend indicators
- **Glucose Features:** Differences, rolling statistics, trends
- **Cumulative Features:** Insulin and carb sums over time windows
- **Outlier Flags:** Binary indicators for anomalous values

3. PyTorch Datasets (`torch_dataset.py`)

OhioDataset

Purpose: Convert preprocessed DataFrames into PyTorch-compatible datasets for deep learning.

Key Features:

- **Sequence Extraction:** Create time series sequences without missing values
- **Standardization:** Z-score normalization using training statistics
- **Insulin Smoothing:** Redistribute extended bolus over time (adapted from GluPred)
- **Flexible Features:** Support univariate (glucose-only) or multivariate models
- **Consistent Normalization:** Share statistics between train/test sets

Usage:

```
from benchmark.data.torch_dataset import OhioDataset,
prepare_personal_data
from torch.utils.data import DataLoader

# Create datasets
train_dataset = OhioDataset(
    raw_df=preprocessed_train_df,
    sequence_length=12,
    prediction_horizon=1,
    unimodal=False # Use multiple features
)

test_dataset = OhioDataset(
    raw_df=preprocessed_test_df,
    sequence_length=12,
    prediction_horizon=1,
    external_mean=train_dataset.mean,
```

```

        external_std=train_dataset.std,
        unimodal=False
    )

# Create DataLoaders
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

# Training loop
for sequences, targets in train_loader:
    # sequences: (batch_size, sequence_length, n_features)
    # targets: (batch_size, prediction_horizon)
    predictions = model(sequences)
    loss = criterion(predictions, targets)
    # ... training code

```

GluPred Integration

This implementation is **based on and inspired by the GluPred project**, with significant adaptations:

From GluPred:

- **Dataset structure:** Core PyTorch Dataset class design
- **Insulin smoothing:** Algorithm for distributing extended bolus over time
- **Sequence extraction:** Method for finding contiguous valid sequences
- **Normalization approach:** Z-score standardization using training statistics

Enhancements for BG-Forecasting:

- **Modular architecture:** Separated loading, preprocessing, and dataset creation
- **Advanced preprocessing:** Comprehensive feature engineering and outlier detection
- **Flexible data handling:** Support for missing values and various data quality issues
- **Research workflow:** Designed for experimental reproducibility

Key Algorithm: Insulin Smoothing

The insulin smoothing algorithm (adapted from GluPred) handles extended bolus delivery:

```

def _smooth_bolus(self, bolus, bolus_dur):
    """Redistribute extended bolus over delivery duration."""
    for i in range(len(bolus)):
        if bolus[i] > 0 and bolus_dur[i] > 0:
            # Find consecutive identical bolus values
            j = 1
            while i + j < len(bolus) and bolus[i + j] == bolus[i]:
                j += 1
            # Distribute total dose over time steps

```

```
        bolus[i:i+j] = bolus[i:i+j] / j
    return bolus
```

This ensures that extended insulin delivery (e.g., for high-fat meals) is represented accurately for ML models.

💡 Usage Examples

Complete Pipeline Example

```
import sys
sys.path.append('benchmark')

from data.loaders import load_ohiot1dm_data
from data.preprocessors import preprocess_ohiot1dm_data
from data.torch_dataset import prepare_personal_data
from torch.utils.data import DataLoader

# 1. Load raw data
train_data = load_ohiot1dm_data(
    data_dir="data/raw/ohiot1dm",
    patient_ids=[540],
    mode='train',
    version='2020'
)

# 2. Preprocess data
preprocessed_data = preprocess_ohiot1dm_data(
    train_data,
    apply_basic_preprocessing=True,
    include_feature_engineering=True,
    normalize=False # PyTorch dataset will handle this
)

# 3. Create PyTorch datasets
train_dataset, test_dataset = prepare_personal_data(
    train_csv_path="processed_data/patient_540_train.csv",
    test_csv_path="processed_data/patient_540_test.csv",
    sequence_length=12,
    unimodal=False
)

# 4. Create DataLoaders
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

# 5. Ready for training!
for epoch in range(num_epochs):
    for sequences, targets in train_loader:
```

```
# Train your model  
pass
```

Multi-Patient Training

```
from data.torch_dataset import prepare_global_data  
  
# Create datasets for transfer learning  
global_dataset, target_train, target_test = prepare_global_data(  
    data_dir="processed_data",  
    patient_id="540",  
    sequence_length=12,  
    unimodal=False  
)  
  
global_loader = DataLoader(global_dataset, batch_size=64, shuffle=True)
```



Research Features

Experimental Reproducibility

- **Deterministic processing:** Consistent results across runs
- **Configurable parameters:** Easy experiment configuration
- **Detailed logging:** Track preprocessing steps and statistics
- **Version control:** Dataset version tracking

Extensibility

The architecture supports:

- **Additional datasets:** Easy integration of new data sources
- **Custom preprocessing:** Modular preprocessing steps
- **Different ML frameworks:** Not limited to PyTorch
- **Various prediction tasks:** Single/multi-step, single/multi-variate



Requirements

Python Dependencies

- `pandas >= 1.3.0`
- `numpy >= 1.20.0`
- `scikit-learn >= 1.0.0`
- `torch >= 1.9.0` (for PyTorch datasets)
- `datetime, xml.etree.ElementTree` (standard library)

Data Requirements

- OhioT1DM dataset XML files in proper directory structure
 - Sufficient disk space for processed data
 - Memory requirements depend on number of patients and sequence length
-

Acknowledgments

This implementation is based on the excellent work from the **GluPred project**:

GluPred: A Glucose Prediction System using Deep Learning

Original authors: [Cui, Ran and Hettiarachchi, Chirath and Nolan, Christopher J and Daskalaki, Elena and Suominen, Hanna]

Our implementation adapts their PyTorch dataset design and insulin smoothing algorithms while extending functionality for comprehensive research workflows.

Citation

If you use this data module in your research, please cite both:

1. **This BG-forecasting project**
 2. **The original GluPred project** that inspired the core dataset design
-

Troubleshooting

Common Issues

1. **Import Errors:** Ensure all dependencies are installed
2. **Missing Data Files:** Check data directory structure
3. **Memory Issues:** Reduce batch size or sequence length
4. **NaN Values:** Check preprocessing configuration

Getting Help

- Check the comprehensive examples in the parent directory
 - Review error messages for specific issues
 - Ensure data files are in the expected format and location
-

Future Enhancements

- **Additional datasets:** Support for more diabetes datasets
 - **Real-time processing:** Streaming data capabilities
 - **Performance optimization:** Faster preprocessing and loading
-

This module provides a robust foundation for blood glucose forecasting research, built on proven algorithms from GluPred and enhanced for modern deep learning workflows.